Inhabiting Choice

Confluent Witnesses, the Phenomena They Cannot Carry,
and One Observation Made Three Times

L. G. Meredith
F1R3FLY.io

November 2026

Abstract

A system whose witnesses reduce confluently can *state* non-confluent phenomena but cannot *inhabit* them: no term of such a system has a branching structure, so the system's identity on its own witnesses is forced to be linear-time, and the logic adequate for that identity is forced to be determinate. This note makes that one observation three times. Abstractly, it is a consequence of Church–Rosser alone: on a confluent witness language the linear-time/branching-time spectrum collapses, whereas on the rho calculus it does not, and the generated Hennessy–Milner logic is obliged to discriminate proofs by the structure of their choices. For software verification, it says that a λ-term-based prover can certify a concurrent system only relative to a stipulated model of its non-determinism, and that the faithfulness of that model, unlike the faithfulness of a deterministic machine model, is not auditable by any means the prover possesses; a prover whose witnesses are processes on a process runtime makes the certificate an inhabitant of the system and the audit an internal bisimulation check. For artificial intelligence, it says that branching-time structure is invisible in any corpus of completed runs — a trace distribution has a unique autoregressive factorisation — so that an intelligence trained only by observation is linear-time by an information-theoretic limitation rather than a design choice, while naturally evolved intelligence demonstrably exploits races as races. The three registers share one spectrum, which we align with Pearl's ladder, and one remedy, which is witnesses that can be races and an identity on them at least as fine as step bisimulation.

1 The observation

The prior notes of this series [1, 2] argued that proof objects in Lean, Coq and Agda — terms of a typed λ-calculus — force proof *search* into an orchestration layer outside the proof dynamics, because confluence leaves no room for choice in the reduction relation and closedness leaves no open end for an incomplete proof to reach out through. The present note isolates what we take to be the underlying fact and states it without reference to search.

> *A system whose witnesses reduce confluently can talk about non-confluent phenomena, but no witness of the system can be one. The phenomena are present in the building and absent from the proofs.*

The consequence that matters is for *identity*. Whatever notion of "same proof" such a system adopts, it is computed from a confluent dynamics, and on a confluent dynamics the distinction between branching-time and linear-time equivalences does not exist. The identity is therefore linear-time whether or not anyone chose it to be, and the logic adequate for it cannot distinguish two witnesses by *when* a choice was made, because no witness ever makes one.

We make the point three times. §2 states it as a pair of propositions about witness languages, with the λ-calculus and the rho calculus [8] as the two poles. §3 reads it for the verification of concurrent software, where it says precisely what a λ-term-based prover's certificate of a concurrent system is a certificate *of*. §4 reads it for artificial intelligence, where it becomes a statement about what can be learned from traces and what requires intervention. §5 lays the three readings side by side, and §6 records what the note leaves owed.

Throughout, the identity relation is the weak context-labelled bisimilarity of the companion papers [3, 4, 5]: labels are minimal enabling contexts in the open form of the Leifer–Milner construction [9], matched up to the relation being defined, with internal steps silent. We write $\approx_W$ for this relation on a witness language W, and $\simeq_W^{\mathrm{tr}}$ for the corresponding weak trace equivalence.

2 The point, abstractly

2.1 Collapse of the spectrum under confluence

Let W be an interactive graph-structured lambda theory in the sense of [6, 5]: a signature, an equational theory, and rewrite rules whose left-hand sides are cuts. Its reduction relation $\to_W$ is the closure of the rules under contexts and equations; its context-labelled transition system has transitions $P \xrightarrow{c} P'$ for minimal enabling contexts c, with the empty context labelling internal steps.

Definition 1 (Determinate)**.** W is *determinate* when $\to_W$ is confluent and internal steps are inert for labelled transitions: if $P \to_W P'$ and $P \xrightarrow{c} Q$ with c non-empty, then $P' \xRightarrow{c} Q'$ for some $Q' \approx_W Q$.

Confluence alone does not give inertness, but for the λ-calculus the two come together: Church–Rosser supplies confluence, and standardisation supplies inertness, since any labelled transition — an application to a fresh argument, in the open label discipline — commutes with internal β-steps up to further internal steps.

Proposition 2 (Collapse)**.** *If W is determinate then, on terms without divergence, $\approx_W = \simeq_W^{\mathrm{tr}}$. The linear-time/branching-time spectrum between them is a single point.*

Proof sketch. This is Milner's determinacy theorem [10] in the labelled setting of Groote and Sellink's τ-confluence [11]. Trace equivalence is contained in bisimilarity on determinate systems because the set of states reachable by a given weak trace is, up to $\approx_W$, a singleton: any two such states are internal-reducts of a common ancestor and reconverge by confluence, and their labelled capabilities agree by inertness. So trace-equivalent terms have bisimilar successors after every label, and trace equivalence is a bisimulation. The reverse inclusion always holds. Divergence is excluded because a divergent term has traces without having the matching derivatives; on the strongly normalising fragment the hypothesis is vacuous. □

Corollary 3 (λ-witnesses)**.** *Let Λ be the λ-calculus as an interactive GSLT with cut application and rule β. On closed, typed, strongly normalising terms, $\approx_\Lambda$ coincides with $\beta\eta$-equality.*

Proof sketch. By Proposition 2, $\approx_\Lambda$ is trace equivalence, and with labels "apply to a fresh argument" traces are applicative behaviour. Two $\beta\eta$-distinct normal forms of the same simple type are separated by an applicative context given two base constants (the typed Böhm theorem [12]), and $\beta\eta$-equal terms are applicatively indistinguishable. On this fragment η is behaviourally invisible, since every closed term of arrow type converges to an abstraction. □

So the proof identity of a λ-witnessed system is *forced* to be $\beta\eta$ on the proof-assistant fragment, and forced to be linear-time everywhere: it was never a choice.

2.2 Non-collapse on the rho calculus

The rho calculus has no choice primitive; non-determinism arises from races on names. Write $\mathsf{race}(a; P, Q)$ for $\mathsf{for}(x \leftarrow a)\, P \mid \mathsf{for}(x \leftarrow a)\, Q \mid a!(\mathbf{0})$, which runs exactly one of P, Q.

Proposition 4 (Non-collapse)**.** *There are closed rho processes P, Q with $P \simeq^{\mathrm{tr}}_{rho} Q$ and $P \not\approx_{rho} Q$, separated by a generated Hennessy–Milner formula.*

Proof. Take the process-calculus textbook pair, rendered with races:

$$P \;=\; \mathsf{for}(x \leftarrow a)\, \mathsf{race}(k;\; b!(\mathbf{0}),\; c!(\mathbf{0})), \qquad Q \;=\; \mathsf{race}(k;\; \mathsf{for}(x \leftarrow a)\, b!(\mathbf{0}),\; \mathsf{for}(x \leftarrow a)\, c!(\mathbf{0})),$$

with k private. Both have weak traces $\{\epsilon, a, ab, ac\}$ under the obvious observers. After P's input on a, both b and c remain reachable; after Q's, exactly one does. The formula $\langle a\rangle(\langle b\rangle\top \wedge \langle c\rangle\top)$, with $\langle \ell\rangle$ the modality for the minimal context offering ℓ, holds of P and fails of Q. By the Hennessy–Milner characterisation [13] they are not bisimilar. □

The pair is familiar; its interest here is what it says about witnesses. Read P and Q as proof-processes of a proposition whose discharge involves choosing between lemma b and lemma c. P is the proof that keeps both options open past the first obligation; Q is the proof that commits before it. They are the *same proof* under any linear-time identity and *different proofs* under bisimulation, and the difference is exactly when the choice was made. No λ-term can be either of them, and no λ-witnessed system can have a proof identity that depends on the timing of a choice, because Proposition 2 says the timing is never there to depend on.

2.3 Talked about and inhabited

The two propositions together are the observation of §1 in formal dress.

Observation 5 (Statement without inhabitation)**.** *A system W* states *a non-confluent phenomenon when some relation definable in W exhibits it — a transition system encoded as data, with P and Q of Proposition 4 as two of its states. W* inhabits *it when some term of W exhibits it under W's own reduction. A confluent W can do the first and never the second. Its identity on witnesses, and the logic adequate for that identity, are therefore blind to the phenomenon even while its propositions quantify over it.*

The generated logic makes the blindness visible. On a determinate W, the generated HML [5] is itself determinate: $\langle c\rangle\varphi \wedge \langle c\rangle\psi$ entails $\langle c\rangle(\varphi \wedge \psi)$ up to internal steps, so the modalities carry no information beyond the trace they name and the possibility of divergence. The adequate formulae for λ-witnesses are a degenerate fragment of the adequate formulae for rho-witnesses, in which the connective that distinguishes P from Q above has nothing to distinguish.

Remark 6 (Which comparison is being made). It is tempting to compare the two systems by encoding Λ into rho and asking whether rho observers separate λ-terms that λ observers identify. The companion paper [4] supplies the discipline for that comparison: an encoding is a pair (F, Φ) on terms and contexts, bisimilarity is indexed by an observer class, and full abstraction holds relative to $\operatorname{im}\Phi$ while the restriction to $\operatorname{im}\Phi$ is forced. Carried out for Λ, the comparison yields three nested relations on λ-terms: relative to encoded λ-contexts, no separation (Conjecture 9 below); relative to encoded π-contexts, the gap between Lévy–Longo tree equality and lazy observational equivalence [14, 15], confined to terms with unsolvable subterms; relative to all rho contexts, a further separation that exposes the encoding's bookkeeping and is uninformative about proofs. That comparison is worth making, and we record its obligations in §6, but it is *not* the point of this note. Every one of those relations is a relation on λ-terms, and every λ-term is confluent. The separation that matters is between the confluent image of F and the non-confluent witnesses outside it, where there is nothing on the λ side to compare against. Observation 5 is about that.

2.4 Beyond branching: steps and causes

The spectrum has a third level above branching time. Ordinary context-labelled bisimulation satisfies the expansion law, $a \mid b \;\approx\; a.b + b.a$ with $+$ rendered by a race, so a rho-witnessed system with that identity inhabits branching but not causal independence — Observation 5 transposed one level up. The remedy is again a matter of presentation rather than of redefining bisimulation. Adding the synchronous rule

$$\frac{P_1 \to Q_1 \qquad P_2 \to Q_2}{P_1 \mid P_2 \;\to\; Q_1 \mid Q_2} \tag{Par-2}$$

to R as a single step — the one-cell shadow of horizontal composition in rewriting logic [18] — makes the derived transition system a step transition system, the induced identity step bisimulation, and the expansion law false. Step bisimulation records simultaneity and not dependence: $a \mid b$ and $(a \mid b) + a.b$ are step bisimilar and pomset-distinct [19]. For dependence the rho calculus has an unusual resource: a reduction creates at most one name, and creates it in a substitution-transparent position [4], so a redex on a name created by an earlier reduction is causally after it and the quote structure of the name is a provenance record. A history-preserving bisimulation for rho can read its dependence relation off name creation on the fresh-name fragment; the rest is the subject of the history-functor note.

2.5 The price

$\beta\eta$ on strongly normalising typed terms is decidable; bisimilarity on a Turing-complete reflective calculus is not. The collapse of Proposition 2 is what buys Lean its decidable proof identity and its cheap checking, and for propositions that are static — inhabitation, arithmetic — it loses nothing, because there is nothing behavioural to lose. The observation bites exactly where the propositions are about behaviour: there a confluent witness cannot carry the branching structure that *is* the content of the proof, and the identity it imposes equates proofs that differ in the one respect the proposition is about.

3 The point, for software verification

3.1 What a λ-term prover does certify

It is important not to overstate the case. Lean, Coq and Isabelle verify non-confluent systems routinely. Iris [20] proves linearisability and race-freedom of fine-grained concurrent data structures; CertiKOS [21] certifies a concurrent operating-system kernel, scheduler included, down to assembly, through a stack of certified abstraction layers and a concurrency-aware compiler whose correctness proof is itself a Coq term. A ticket-reservation system's "no double booking under any interleaving" is a universally quantified statement over a transition relation, and its proof is a λ-term doing induction on reachable states with an invariant. Nothing in §2 prevents this: the non-confluence is *stated*, as an inductive relation, and the proof is about the relation.

So the claim is not that such systems cannot be verified with λ-term witnesses. It is a claim about what the certificate is a certificate of, and about which part of the trusted base can be examined.

3.2 What is outside: the model of non-determinism

Every such development begins by stipulating a model of the non-determinism: the interleaving semantics, the scheduler, the memory model, entered as a definition of which executions exist. The proof quantifies over that definition. The faithfulness of the definition to the machine is

not a theorem anywhere; it is the trusted base, as the instruction-set model is the trusted base of a sequential verification. The asymmetry between the two cases is the point.

Observation 7 (Unauditability)**.** *For a sequential system the trusted base is a deterministic step function, and its faithfulness is auditable by execution: run the machine, run the model, compare traces, and every discrepancy is a counterexample found by running. For a concurrent system the trusted base is a relation — a set of permitted interleavings — and no finite set of observed runs distinguishes it from many other relations. Faithfulness is a branching-time property of the model, and checking it requires a branching-time observer. By Observation 5, a λ-term-based prover contains no inhabitant with branching structure, hence nothing that can be run against the machine to perform the check.*

The trust is therefore not merely unproven; it is unexaminable by any means the prover possesses. The sequential verifier's trusted base is auditable by the one thing a λ-prover does have, execution. The concurrent verifier's is not.

3.3 Cost, and where the certificate lives

Two further consequences follow, both visible in the heroic projects.

First, cost. In the λ-witnessed world the certificate is a term checked once and discarded, and the running system is a different object produced by extraction or compilation. The link between them is a theorem — a verified compiler whose correctness statement is a simulation under all interleavings — and at every refinement layer the branching structure the witness cannot carry must be re-encoded by hand as a relation and the refinement re-proved as a λ-term *about* two relations. CertiKOS reaches the bottom of its stack at a cost proportional to exactly the structure its proof objects cannot inhabit, and the next concurrent system starts over.

Second, deployment. A λ-proof cannot be run against the system it certifies. A rho-witnessed proof with pending obligations *is* a process, — under the obligation-as-communication reading of [1] it is a runtime monitor — and on a runtime that is the calculus, context-labelled bisimulation quantifies over the environments the system will actually meet, because the contexts *are* rho processes. The model/implementation gap does not vanish; it moves, from "this model is faithful to the machine" (per system, unauditable) to "this implementation is faithful to the calculus" (once, for the platform). That is what verified compilation did for sequential code. Process witnesses do it for concurrent code by construction rather than by a per-system refinement stack.

3.4 True concurrency makes the gap wider

The interleaving models of §3.1 are a defensible projection on a single core or under sequential consistency. On multicore hardware they are false, and the field models weak memory axiomatically, as partial orders on events per candidate execution [22], because interleaving cannot express the reorderings the hardware permits. The trusted base is then a partial-order model, and Observation 7 sharpens: its faithfulness is a property of an independence relation, two levels above anything a confluent witness can inhabit. On the rho side the corresponding move is Par-2 of §2.4; the F1R3FLY runtime is itself a partial-order system, a block DAG merging independent deploys by conflict detection, so a verification on that runtime whose identity is interleaving bisimulation under-specifies the system it runs on. For the ticket system the difference is concrete: two reservations on disjoint seats should commute, an interleaving proof reaches that fact only through a linearisability argument that re-derives commutation from outside, and a step-concurrent witness has the commutation as a single Par-2 step.

4 The point, for artificial intelligence

4.1 Traces cannot reveal branching

The value proposition of the current programme is that the salient aspects of intelligence are software, and in its dominant form, that they are what one obtains by predicting traces at scale. Proposition 4 has a direct consequence for that form.

Proposition 8 (Observational invisibility of branching)**.** *Let μ be a probability distribution over finite traces. Then μ has a unique autoregressive factorisation $\mu(x_1 \cdots x_n) = \prod_i \mu(x_i \mid x_{<i})$, and any two labelled transition systems inducing μ — in particular the probabilistic renderings $p \cdot a.b + (1-p) \cdot a.c$ and $a.(p \cdot b + (1-p) \cdot c)$ of Proposition 4 — induce the same factorisation. A learner whose only access to the system is samples from μ cannot distinguish them.*

Proof. Conditional probabilities are determined by the joint distribution; the two systems are trace-distribution equivalent by construction and not probabilistically bisimilar [23]. □

So a next-token predictor trained on a corpus of completed runs, however large, represents the trace distribution and nothing more. The question "was the die cast before or after a?" has no answer in the data. The limitation is not architectural; it is information-theoretic. *Branching-time structure is invisible to any observer that does not act.*

4.2 Experimenters and spectators

To learn branching structure one has to intervene: place the system in a state, supply a context, and observe what it can still do. That is the context-labelled bisimulation game of [3, 5] read as an epistemology — the observer chooses a label, the system answers — and the mortal-scientist programme [7] is its agent. Bisimulation is the equivalence an *experimenter* can learn; trace equivalence is what a *spectator* gets.

The alignment with Pearl's ladder [24] is exact. Rung one, association, is linear time: what a trace distribution supports. Rung two, intervention, is branching time: what an experimenter supplying contexts can learn, and what bisimulation characterises. Rung three, counterfactuals, requires the causal structure, which is the true-concurrency level of §2.4 — history-preserving bisimulation, with dependence as an object. The ladder and the linear/branching/causal spectrum are one hierarchy seen from statistics and from concurrency theory.

4.3 Where the stack sits

The indictment is specific and should be aimed carefully. Next-token prediction on observational text is rung one by construction. Reinforcement learning is not: a Markov decision process is a branching-time object, a state with actions and distributions over successors, and bisimulation for MDPs is an established tool for state abstraction [25]. An RL agent learns by intervening. Post-training a language model with outcome rewards is the community purchasing rung-two data for a rung-one model, which is evidence that the limitation is felt.

But reinforcement learning stops one rung short of nature. An MDP, and even a Markov game with simultaneous moves, has a global clock: the world steps in lockstep. The world does not. The races of the natural world — which sperm fertilises the egg, which antelope the pride culls, which flower the bee reaches — are truly concurrent, and an agent that models them as interleaved under a global tick attributes dependence where there is only coincidence. That is the step/pomset gap of §2.4, now as a claim about what evolved intelligence must track.

4.4 Nature exploits races as races

The examples must be of the right kind. Sensitivity to *who won* a race is observation of an outcome, rung one. The claim requires organisms sensitive to the race *as a race* — that it could go otherwise, and that one can act on the branching. The ethological and evolutionary literature supplies them.

- **Protean behaviour.** Prey move unpredictably in order to defeat a predator's predictive model [26]. The strategy only makes sense against an opponent reasoning in branching time, and only works if the prey itself can generate genuine branching.
- **Mixed strategies.** Hawk–dove and every mixed evolutionarily stable strategy [27] is a population exploiting the fact that the opponent must model a choice, not a trace.
- **Bet-hedging.** Organisms produce offspring carrying non-confluent strategies, so that whichever environment arrives some win [28]. This is the biological hedge: pursue both branches, keep what succeeds.
- **Sexual selection.** The race is manufactured; the ornament exists to structure a competition whose outcome is informative only because it was a race.

The lion that drives the herd to *create* a race it can win is doing rung-two work; the lion that eats the slowest is doing rung one. The first is intelligence; the second is a filter.

4.5 Witnesses, again

The last step returns to §2. An agent's knowledge is its hypotheses and plans — in the mortal-scientist setting, its proofs. Bet-hedging is a *plan that is itself a race*: pursue two branches in parallel and commit to whichever lands. That plan is a non-confluent witness. A λ-plan cannot be it; it can describe it from outside and hand it to a scheduler, which is the orchestration-outside-the-dynamics of [1] reappearing as a limit on cognition. So the requirement on a software rendering of intelligence is not only that it *discriminate* branching phenomena but that its internal representations *inhabit* them. Observation 5, drawn through the agent's own cognition: an intelligence whose hypotheses are confluent can observe hedging but cannot hedge in its hypotheses.

5 One point, three registers

	abstract	**verification**	**intelligence**
the confluent object	a λ-witness	a proof about a model	a trace predictor
what it can state	a transition relation	the system's interleavings	the world's runs
what it cannot inhabit	a race	the system's nondeterminism	a hedge
forced identity	$\beta\eta$ / trace eq.	model-relative certificate	trace distribution
what is unexaminable	timing of choice	faithfulness of the model	when the die was cast
the remedy	rho witnesses, $\approx_{\mathsf{rho}}$	process certificates on a process runtime	an experimenter with race-shaped hypotheses
one level up	Par-2, provenance	weak memory, block DAGs	clockless nature, rung three

Table 1: The observation in its three readings.

The rows of Table 1 are not analogies. The same proposition — collapse of the spectrum under confluence — underlies all three columns, and the same remedy answers all three: witnesses that can be races, and an identity on them at least as fine as step bisimulation, with the generated logic following the presentation. The verification column and the intelligence column are the abstract column applied to a certificate and to a hypothesis respectively; a certificate is a hypothesis one has stopped testing.

6 Obligations

The note is an observation with consequences, and it leaves theorems owed.

Obligation 1 (Determinacy of Λ)**.** Establish inertness for the open minimal-label transition system of Λ under the BIHOL construction, so that Proposition 2 applies without the standardisation hand-wave, and identify $\approx_\Lambda$ exactly on untyped terms: lazy observational equivalence if the labels are closed second-order contexts [16], normal-form bisimilarity — Lévy–Longo tree equality [15] — if they are open with fresh variables. The two differ on terms with unsolvable subterms and nowhere else.

Conjecture 9 (Full abstraction relative to encoded contexts)**.** *For the composite encoding* $[\![-]\!] : \Lambda \to \mathsf{rho}$ *through Milner's encoding [17] and the local-allocator encoding of [4],* $M \approx_\Lambda N$ *iff* $[\![M]\!] \approx_{\mathsf{rho},\mathrm{im}\,\Phi} [\![N]\!]$*. The* π*-to-rho half is the full-abstraction theorem of [4]; the* λ*-to-*π *half is operational correspondence for the standard encoding, with the base case of soundness the place to watch.*

Obligation 2 (Step and causal identities)**.** Verify that Par-2 as a rule scheme admits redex-RPOs in the sense of [3], so that step labels are multisets of minimal contexts and step bisimulation is the derived identity; then align the provenance relation read off name creation with history-preserving bisimulation on the fresh-name fragment, and state what the history functor adds on the remainder.

Obligation 3 (The audit as a theorem)**.** Observation 7 is stated informally. Its content is that faithfulness of a non-deterministic model is not a trace property, which is Proposition 8 read for machines rather than corpora; a precise statement should say which class of tests a λ-prover can run against a system and prove that the class is contained in trace observation.

Obligation 4 (Target 3 of [1], relocated)**.** "Proof identity is bisimulation" is trivial on saturated proof-processes, which have no transitions and are therefore all bisimilar to $\mathbf{0}$. The target should be restated on open proof-processes — proofs as functions of their obligations — where §2.1 says it agrees with $\beta\eta$ on the λ-image and §2.2 says it does not collapse outside it; exported witnesses, compared as names, are identified up to structural congruence and belong to a separate proposition.

Acknowledgements

The questions that shaped this note — whether λ-witnessed systems are even cognizant of non-confluent phenomena, what the functional difference is for a verifier armed only with such a prover, and whether the argument survives true concurrency — were put to the author in conversation and are answered here in the order they were asked. Claude (Anthropic) assisted with source review and drafting.

References

[1] L. G. Meredith. Proof search as concurrent reaction: the rho calculus and the category of continued interactive GSLTs as a substrate for proof objects with internalized, cost-accounted search. F1R3FLY.io technical note, 2026.

[2] L. G. Meredith. Proofs as processes, continued. F1R3FLY.io technical note, 2026.

[3] C. B. Wells, M. Stay and L. G. Meredith. Behavior in higher-order languages: admissible observers, relative pushouts, and the encoding of extended theories. In preparation, F1R3FLY.io, 2026.

[4] L. G. Meredith. Namespaces without a name server: freshness by communication, and a fully abstract encoding of the π-calculus in the rho calculus. F1R3FLY.io technical note, v3, 2026.

[5] L. G. Meredith. Spatial formulae, discrimination power, and the observer extension of an interactive GSLT. F1R3FLY.io technical note, v2, 2026.

[6] L. G. Meredith. Graph-structured lambda theories: a reference for the working developer. F1R3FLY.io, 2026.

[7] L. G. Meredith. The mortal scientist: learning as cost-accounted computation in a reflective higher-order calculus. F1R3FLY.io, 2026.

[8] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science* **141**(5):49–67, 2005.

[9] J. J. Leifer and R. Milner. Deriving bisimulation congruences for reactive systems. In *CONCUR 2000*, LNCS **1877**, pp. 243–258. Springer, 2000.

[10] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[11] J. F. Groote and M. P. A. Sellink. Confluence for process verification. *Theoretical Computer Science* **170**(1–2):47–81, 1996.

[12] R. Statman. λ-definable functionals and $\beta\eta$ conversion. *Archiv für mathematische Logik und Grundlagenforschung* **23**:21–26, 1983.

[13] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM* **32**(1):137–161, 1985.

[14] D. Sangiorgi. The lazy lambda calculus in a concurrency scenario. *Information and Computation* **111**(1):120–153, 1994.

[15] S. B. Lassen. Bisimulation in untyped lambda calculus: Böhm trees and bisimulation up to context. *Electronic Notes in Theoretical Computer Science* **20**:346–374, 1999.

[16] P. Di Gianantonio, F. Honsell and M. Lenisa. RPO, second-order contexts, and λ-calculus. *Logical Methods in Computer Science* **5**(3), 2009.

[17] R. Milner. Functions as processes. *Mathematical Structures in Computer Science* **2**(2):119–141, 1992.

[18] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science* **96**(1):73–155, 1992.

[19] R. J. van Glabbeek and U. Goltz. Refinement of actions and equivalence notions for concurrent systems. *Acta Informatica* **37**:229–327, 2001.

[20] R. Jung, R. Krebbers, J.-H. Jourdan, A. Bizjak, L. Birkedal and D. Dreyer. Iris from the ground up: a modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming* **28**, 2018.

[21] R. Gu, Z. Shao, H. Chen, X. Wu, J. Kim, V. Sjöberg and D. Costanzo. CertiKOS: an extensible architecture for building certified concurrent OS kernels. In *OSDI 2016*, pp. 653–669. USENIX, 2016.

[22] J. Alglave, L. Maranget and M. Tautschnig. Herding cats: modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems* **36**(2), 2014.

[23] K. G. Larsen and A. Skou. Bisimulation through probabilistic testing. *Information and Computation* **94**(1):1–28, 1991.

[24] J. Pearl. *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.

[25] R. Givan, T. Dean and M. Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence* **147**(1–2):163–223, 2003.

[26] D. A. Humphries and P. M. Driver. Protean defence by prey animals. *Oecologia* **5**:285–302, 1970.

[27] J. Maynard Smith. *Evolution and the Theory of Games*. Cambridge University Press, 1982.

[28] J. Seger and H. J. Brockmann. What is bet-hedging? *Oxford Surveys in Evolutionary Biology* **4**:182–211, 1987.